Meet your next little sidekick.

Lil Token · Open Source Build Kit

Build the talking desk robot from the video, explore how the agent works, or borrow one piece for a project of your own.

A TINY ROBOT. YOUR NEXT PROJECT.

Lil Token

Build your own little sidekick.

The code, the faces, and how it all fits together.

OPEN SOURCE BUILD KIT



The source, the faces, the architecture and the failures that shaped the build.

Start with the offline example. Then take the hardware route when you are ready. Your own keys, voice and device settings go in local configuration.

Open the public repository

Choose a starting point

You can learn something from this kit before plugging in a robot.

Explore the agent

Run the offline example to watch a question become a tool request and an answer. It uses invented calendar data and the Python standard library.

```
git clone https://github.com/promptadvisers/lil-token-robot.git
cd lil-token-robot
python3 examples/desk_demo.py
```

Make him yours

Open assets/faces/. The fourteen expression, eye and mouth PNGs are ready to edit. The generators and the binary face sets are included.

Build the whole robot

Bring an M5Stack StackChan, a Mac, a USB data cable and a private Wi-Fi network. Set up your own Claude and Fish Audio credentials. Search and account tools can be added later.

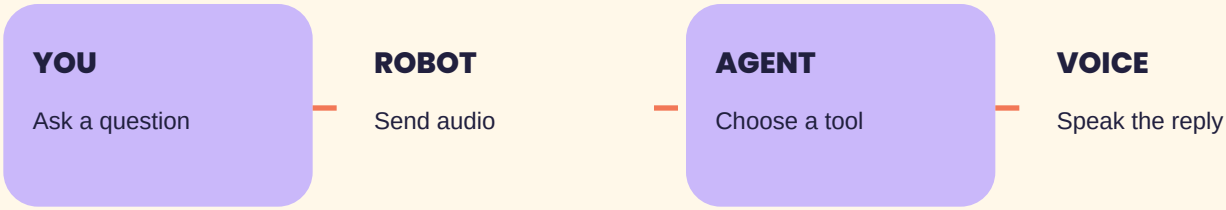
Give your AI a useful job

The prompts folder contains separate prompts for hardware research, the first conversation loop and repeatable testing. Fill in your device and operating system, then begin with the research prompt.

Read next: quick start or face guide

One question, end to end

The robot supplies the physical senses and expression. The computer carries the conversation and calls the services you configure.



Local audio handling → configured cloud services → the robot speaker

Listen

The microphone sends audio through the gateway. Local speech recognition transcribes it and checks for the wake phrase. The intentional request goes to the agent.

Choose

The Claude session sees the question and available tool definitions. It can request a calendar read, a search or a camera view. Python runs the selected tool and returns its result.

Speak

The agent writes a short response. Completed sentences enter the speech queue. Fish Audio generates speech using your selected voice, and the gateway streams it to the speaker.

React

Sensor events can change a face or trigger a short line without asking the language model to decide every movement. A head pat can interrupt speech.

See the full architecture and data flow

Get the software ready

Use Python 3.12, Git and uv. On a Homebrew Mac, install libopus with brew install opus. Firmware builds additionally need Docker.

```
bash tools/bootstrap-src.sh
uv venv .venv --python 3.12
uv pip install --python .venv/bin/python -r brain/requirements.txt
```

Install the gateway and its audio extras from the reconstructed source. The exact command is in docs/QUICKSTART.md.

```
uv pip install --python .venv/bin/python -e \
  'src/gateway[tts-elevenlabs,stt-faster-whisper,stt-openai]'
.venv/bin/python tools/configure.py
```

Fill in your local .env

Add ANTHROPIC_API_KEY, FISH_AUDIO_API_KEY and STACKCHAN_FISH_VOICE_OWNER. configure.py creates a unique device token. Keep it private.

Set HOST and VISION_HOST to your Mac’s private LAN address when the robot is ready to connect. Keep MCP_HTTP_HOST at 127.0.0.1. Set LILTOKEN_TIMEZONE to your local IANA timezone.

Check the chosen models

The source retains the recorded build’s drama-3 speech setting. Your Fish account may require a different available model. The setup uses supported API credentials and your own permitted voice.

```
.venv/bin/python tools/run.py doctor
```

Open the complete setup instructions

Bring the robot online

Make recovery possible

Identify the exact board, flash size and serial device. Make and verify a full private backup before changing firmware. Device backups can contain Wi-Fi settings.

Build from a fixed source

bootstrap-src.sh reconstructs the pinned upstream source and the public patch. Review the generated firmware build and flashing instructions for your exact hardware. Keep the partition table, app and assets consistent.

Connect your network

Set the robot’s Wi-Fi network, the gateway URL on your Mac’s LAN address and the same device token as your .env. Keep device-facing ports on the trusted local network.

```
.venv/bin/python tools/run.py gateway
# In a second terminal
.venv/bin/python tools/run.py brain
```

Let the local transcription model finish its first download. Check the process, then try “Hey Lil Token, say hello.”

```
curl http://127.0.0.1:8771/health
```

Check the whole loop

Listen for your chosen voice. Ask a follow-up. Pat the head during a reply. Ask it to look at an object. Test one action at a time and keep the terminal errors visible.

Read the firmware and recovery guide

Give him some personality

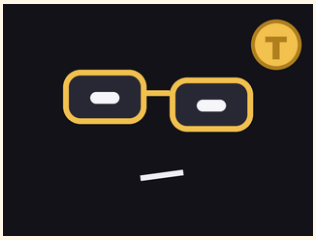
The face uses bold shapes because tiny details disappear on a small display. Expressions, blinking and speech each have their own frames.



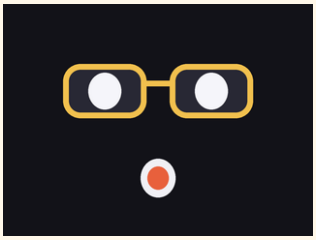
idle



happy



thinking



surprised

Edit the PNGs

The six expression files are idle, happy, thinking, sad, surprised and embarrassed. Three eye states and five mouth states complete the set.

```
.venv/bin/python avatar/make_face.py
.venv/bin/python avatar/make_party.py
.venv/bin/python avatar/make_wardrobe.py
```

The first command creates frames. The others generate the runtime face sets. Research and app badges use make_research.py with librsvg installed.

Connect a face to an event

Open brain/senses.py. A pickup event maps to surprise and one short reaction. Try changing a single line before redesigning the character.

Respect the file format

Layered sets contain fourteen 160×120 RGB565 images, totaling 537,600 bytes. An overlay contains color pixels plus alpha, totaling 57,600 bytes. Check the byte count before transferring it.

See face assets, order and device commands

Build in useful layers

Research the hardware

Understand the existing firmware and the parts it controls. Keep a source list and a recovery plan. The first prompt in the kit stops at the plan.

Prove one conversation

Join microphone, transcription, agent, speech generation and playback. Each layer should work separately before you debug the whole chain.

Add the first useful tool

Use synthetic calendar data first. Then configure a read-only test account. Check that the model called the tool and that its answer matches the returned data.

Make waiting and interruption work

Queue complete sentences, keep one owner of the speaker and let a head pat stop playback. Cache short repeated speech by text, voice and model.

Add the body

Map touch and motion to expressions. Introduce music classification after ordinary speech works. Fans and motor noise are part of the test set.

Test without standing there all day

Play permitted synthetic voices through your speaker. Record input, expected behavior, actual behavior and failure time. Fix one layer, freeze the change and test fresh examples.

Read the longer build guide

When it goes wrong

It wakes when nobody called it

Inspect the wake matcher, greeting rules and negative phrases. Include television speech, “a little bit” and the robot’s own speaker audio.

It ignores your head pat

Check touch-state reads and sensor initialization. The build includes reinitialization after repeated failures and a motion-sensor reset on boot.

It will not stop dancing

Motor noise can look like music. Check background energy, sustained sound and cooldown. Test the stop phrase while the servos are moving.

It opens the camera for email

“Look at my inbox” and “look at this mug” need different tools. Inspect the actual tool call and add both phrases to the routing tests.

It takes too long to answer

Measure transcription, model, tool, synthesis and playback time separately. Check cold model loading and face downloads before changing the prompt.

The face is corrupted

Check dimensions, RGB565 byte order and the expected payload size. A larger camera resolution can also break the live preview path.

Open the full troubleshooting catalogue

Connect your own tools

Turn on one integration at a time. Keep a test account until you have checked its behavior.

Search and calendar

Firecrawl uses your search key. Google uses your own short-lived access token and read scopes. Its public adapter reads Calendar, Gmail and Drive.

Other apps

Composio starts off. A read-only session can expose the tools you choose. Its optional write mode needs extra care because spoken-intent matching cannot guarantee that every risky request is blocked.

Music, camera presence and voice

Install the optional dependencies for local audio classification and presence. Download the required models from their official sources. Use only voices and training audio you are entitled to use.

Code jobs

The implementation is included, off by default, with an empty allowed-folder map. Experiment in a disposable checkout. Tests can execute code, so an allowed-folder list is not a complete sandbox.

Know what leaves the computer

Active requests and selected tool results go to Claude. Speech text goes to Fish. Search queries go to Firecrawl. Requested camera understanding sends an image to the model. Memory and enrollment state stay local.

Read integration and privacy details

Your next little experiment

Pick one change that makes the robot more useful or more yours. Give it a new reaction. Make it read a test calendar. Change a hat. Keep the first experiment small enough to verify.

The files to keep handy

README.md is your entry point. docs/QUICKSTART.md holds the run commands. docs/ARCHITECTURE.md explains the flow. docs/FACES.md explains the art. docs/TROUBLESHOOTING.md holds the failure catalogue.

The prompts

01-research-and-backup.md identifies the hardware and plans recovery. 02-build-the-agent.md builds a single working loop. 03-test-and-improve.md turns mistakes into repeatable tests.

The release boundary

The public source uses portable configuration and account features start off. Your credentials, voice profile and device backup belong in your own private setup. The automated tests use stubs for hardware and cloud calls. Recheck the full loop on your device.

The people and projects behind it

Mark Kashef made the video and experiment. The implementation builds on stackchan-mcp and xiaozhi-esp32. Fish Audio sponsored the video and provides speech synthesis. Third-party licenses and model notices stay with their components.

Open Lil Token on GitHub

Build more projects with the Early AI Adopters community

Public build kit · 12 November 2026